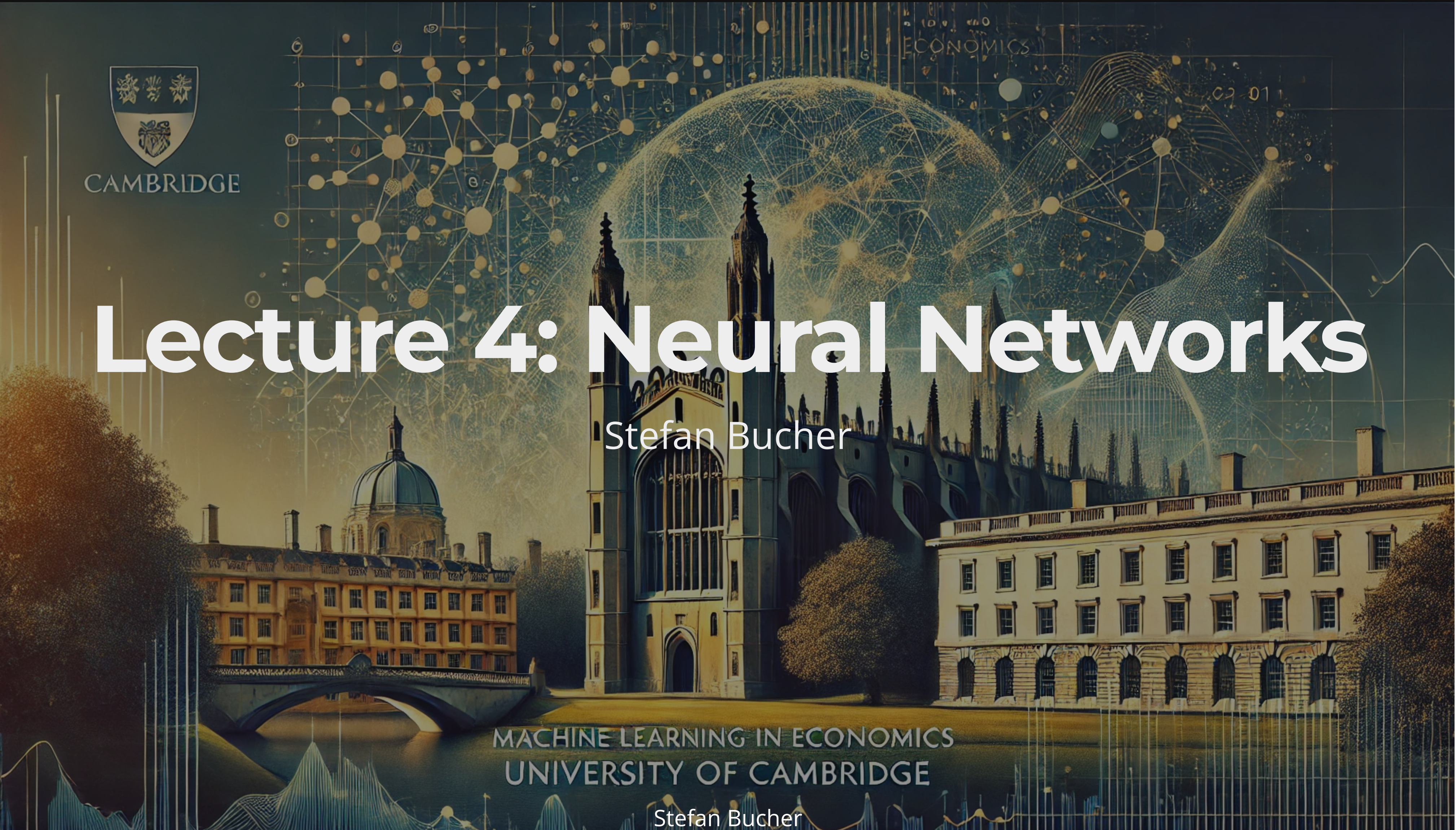


Lecture 4: Neural Networks

Stefan Bucher

Open in Colab

Prince (2023, chaps. 3, 4, 7).[1]

1. Figures taken or adapted from Prince (2023). All rights belong to the original author and publisher. These materials are intended solely for educational purposes.

Shallow Neural Networks

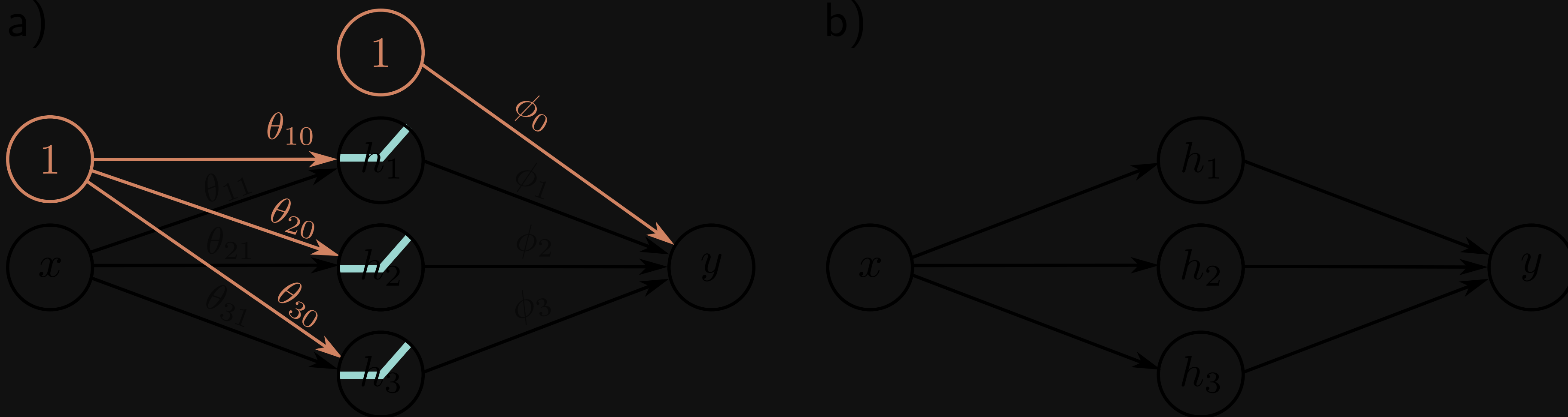
Prince (2023, chap. 3)

Example

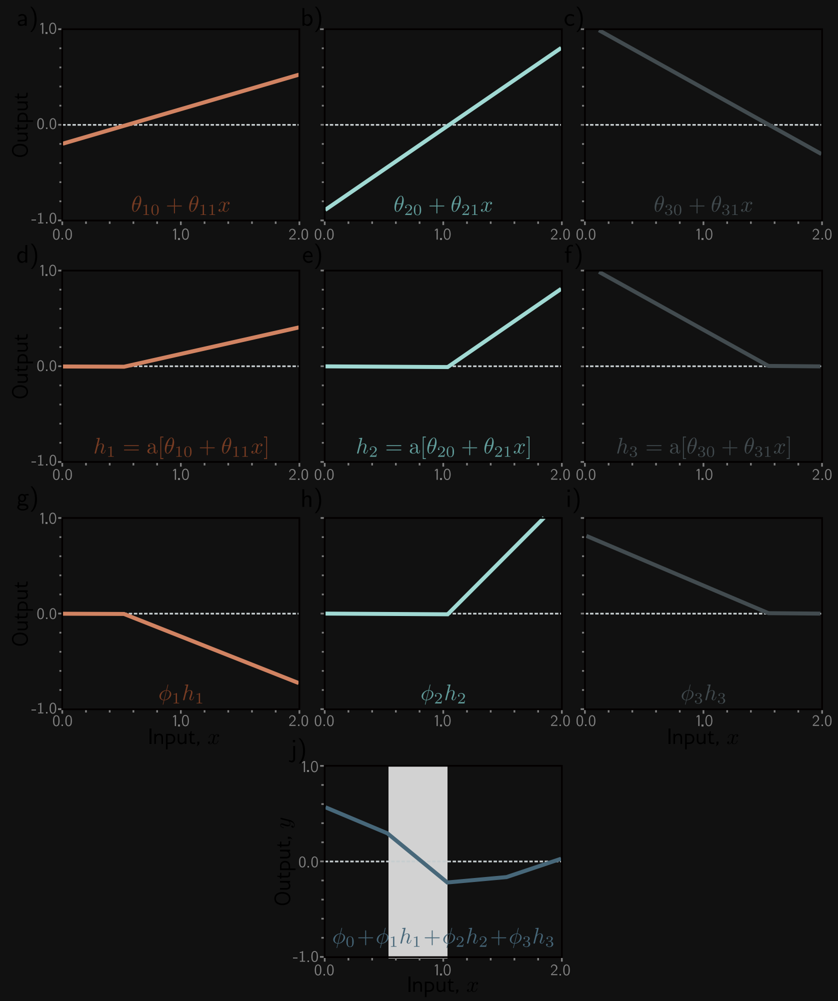
$$f[x,\phi] = \phi_0 + \phi_1 \underbrace{a(\theta_{10} + \theta_{11}x)}_{h_1} + \phi_2 \underbrace{a(\theta_{20} + \theta_{21}x)}_{h_2} + \phi_3 \underbrace{a(\theta_{30} + \theta_{31}x)}_{h_3}$$

$$a(z) = ReLU(z) = \max\{z, 0\}$$

Neural networks (shallow & deep!) with ReLU represent continuous piecewise linear functions.

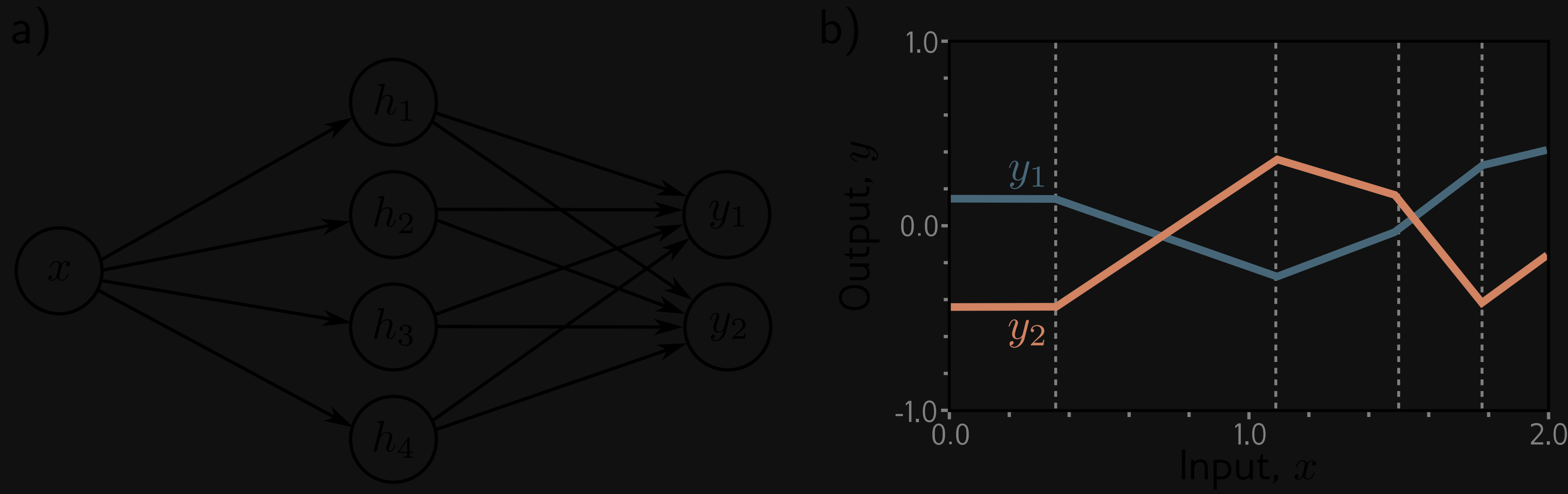


Activation Patterns



Joints where one hidden unit becomes (in)active.

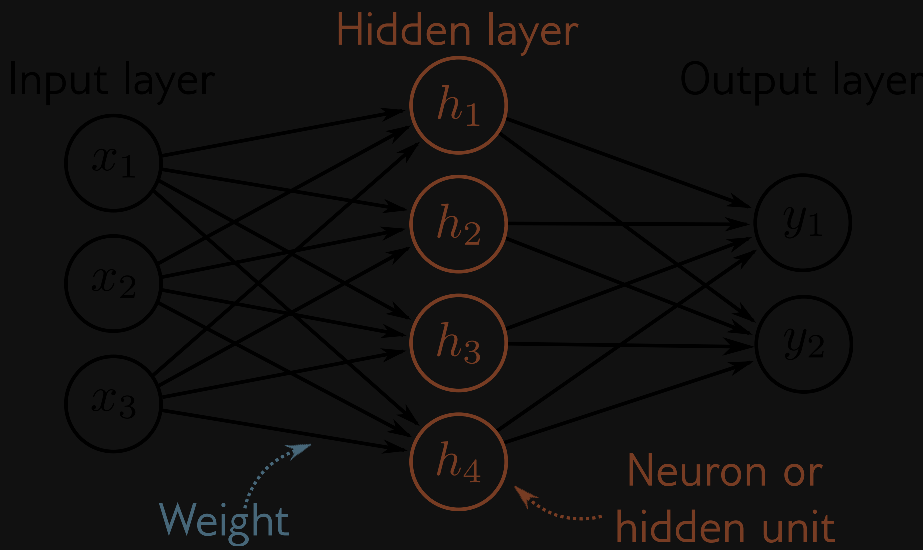
Multivariate Output



Shallow Neural Networks

Map n_{in}-dimensional input to n_{out}-dimensional output.

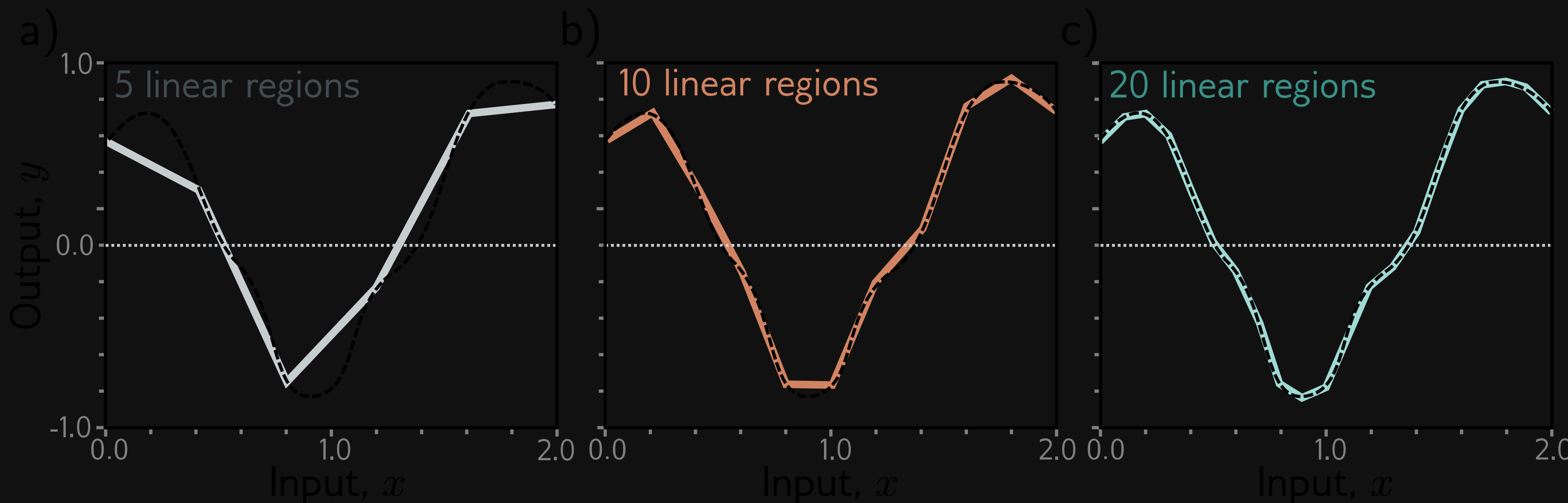
$$y_j = \phi_{j0} + \sum_{d=1}^{D} \phi_{jd} \underbrace{a\left(\theta_{d0} + \sum_{i=1}^{n_{in}} \theta_{di} x_i\right)}_{h_d}, \; j = 1, \ldots, n_{out}$$



Shallow Networks (3.3 & 3.8)

Universal Approximation Thm

$\exists$ shallow neural network with a single layer of sufficiently many hidden units that can approximate, with arbitrary precision, any continuous function from a compact subset of $R^{n_{in}}$ to $R^{n_{out}}$.



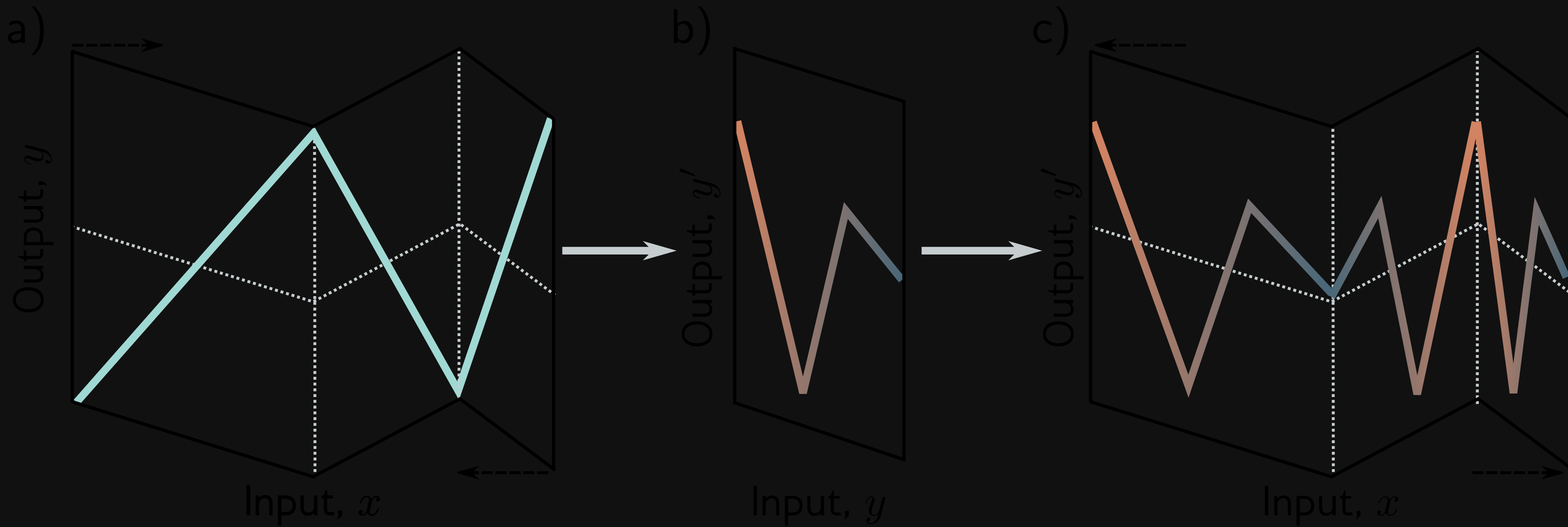
Deep Neural Networks

Prince (2023, chap. 4)

Why Depth? (4.1)

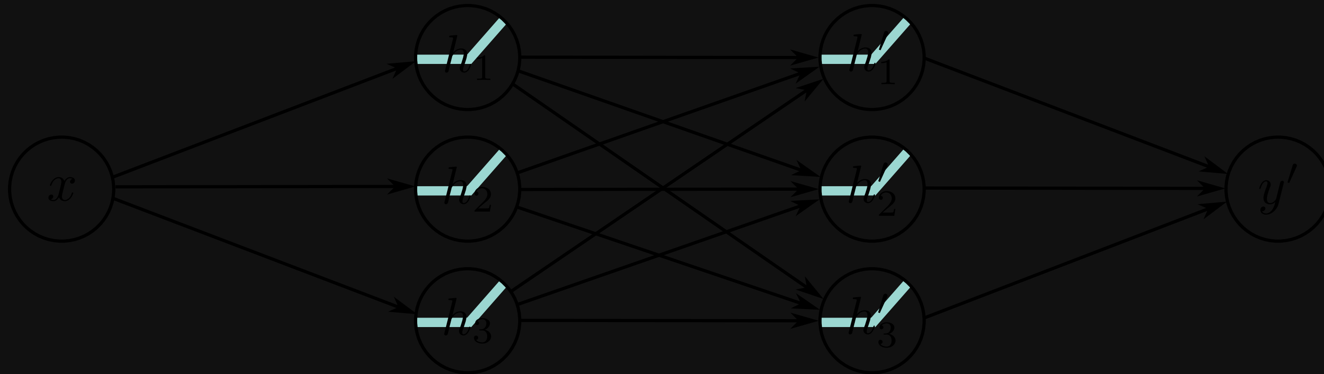
- Universal approximation theorem...
- Deep: more linear regions for same # parameters

Extra Layer is Folding

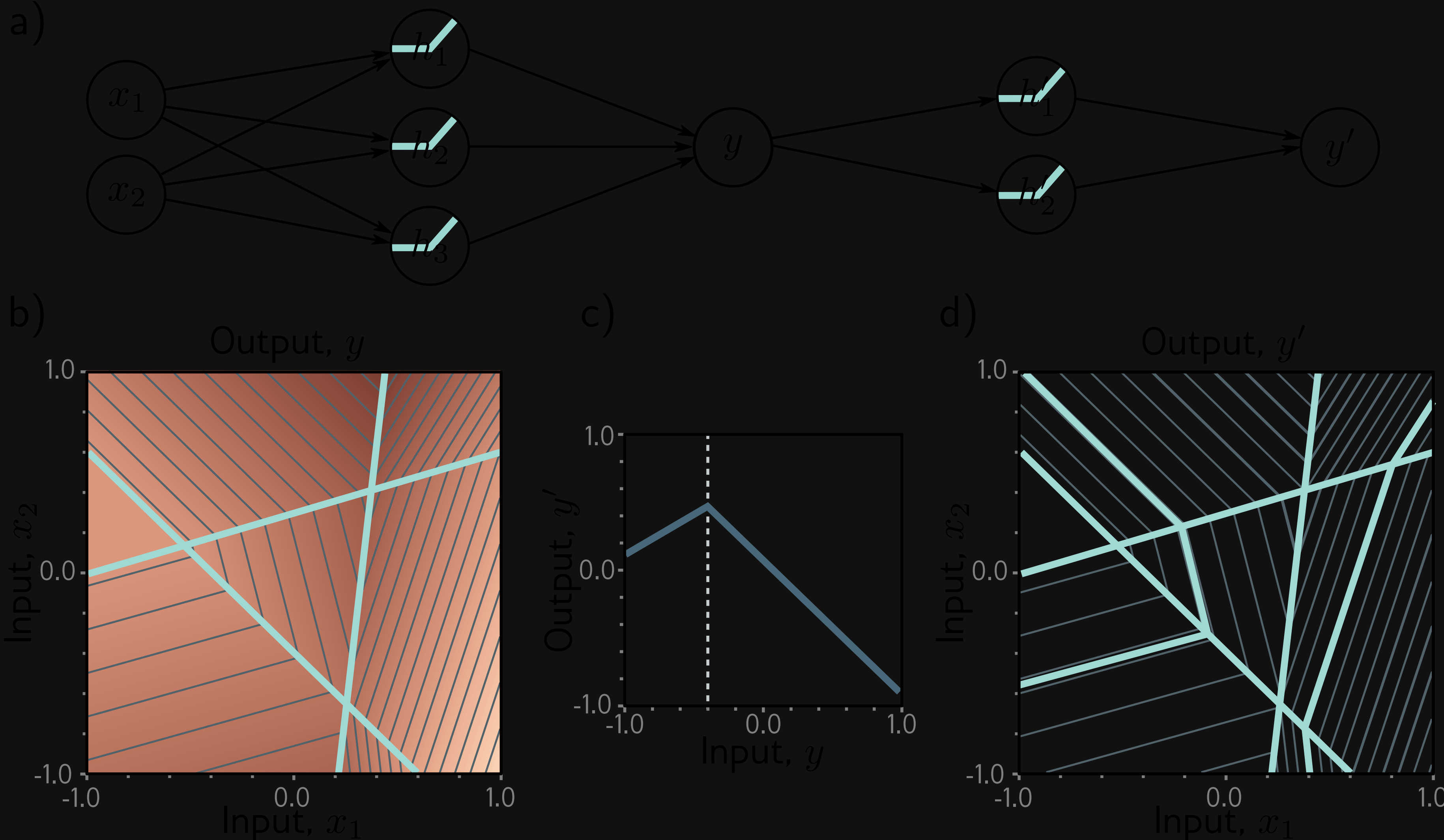


Extra Layer is Clipping (4.5)

Composing as Special Case

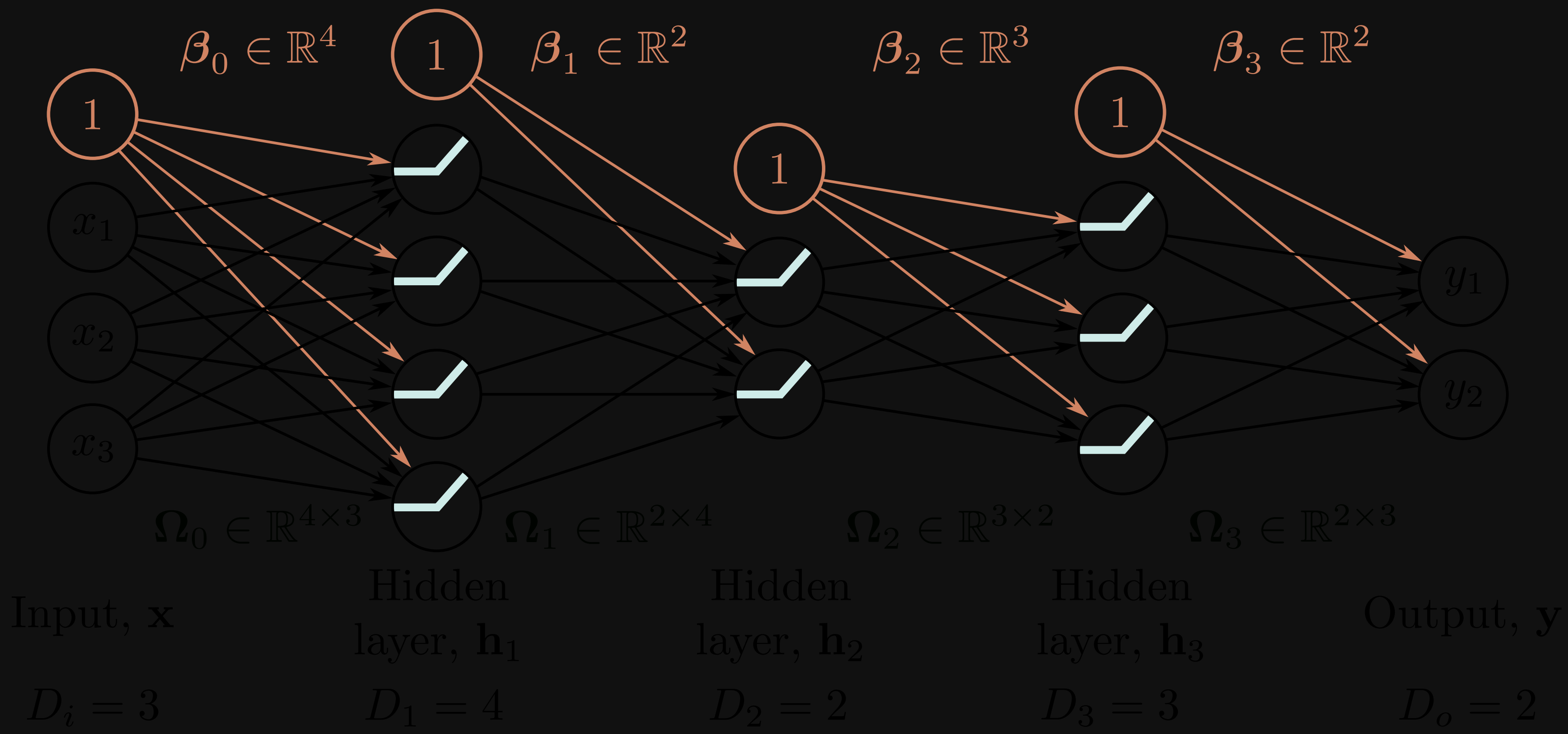


Composing in 2D

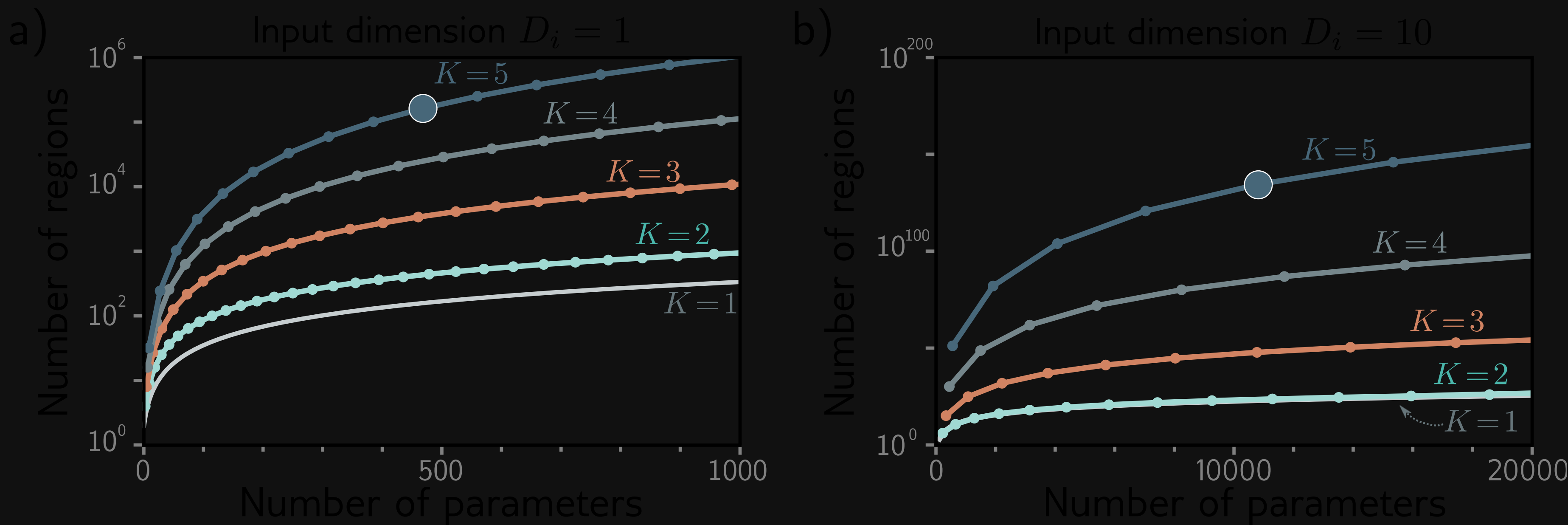


Deep Neural Network

$$\mathbf{h}_1 = \mathbf{a}(\beta_0 + \mathbf{\Omega}_0 \mathbf{x})$$
$$\mathbf{h}_k = \mathbf{a}(\beta_{k-1} + \mathbf{\Omega}_{k-1} \mathbf{h}_{k-1}), \quad k = 2, \ldots, K$$
$$\mathbf{y} = \beta_K + \mathbf{\Omega}_K \mathbf{h}_K$$



Depth Efficiency



Backpropagation

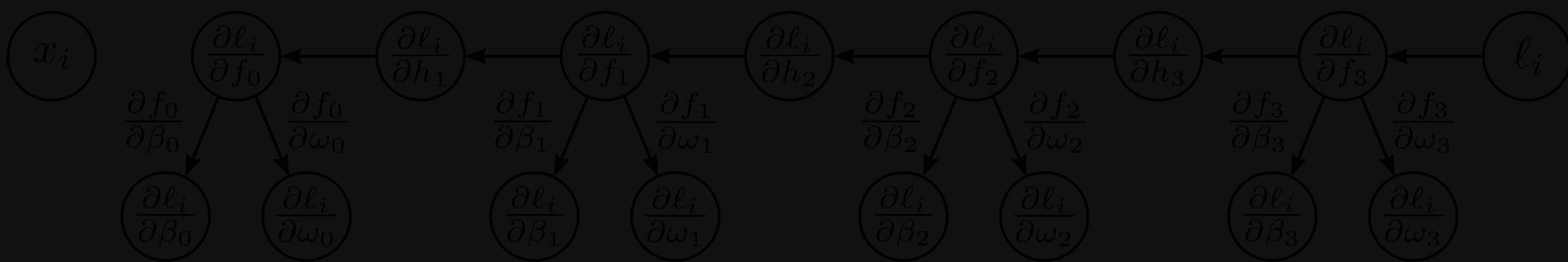
Prince (2023, chaps. 7.1–7.4)

Computing Gradients with Chain Rule

Stochastic Gradient Descent

$$\phi_{t+1} \leftarrow \phi_t - \alpha \sum_{i \in B_t} \nabla_\phi l_i(\phi_t)$$

requires computing $\frac{\partial l_i}{\partial \beta_k}$ and $\frac{\partial l_i}{\partial \boldsymbol{\Omega}_k}$ at each layer $k \in \{0, \ldots, K\}$.

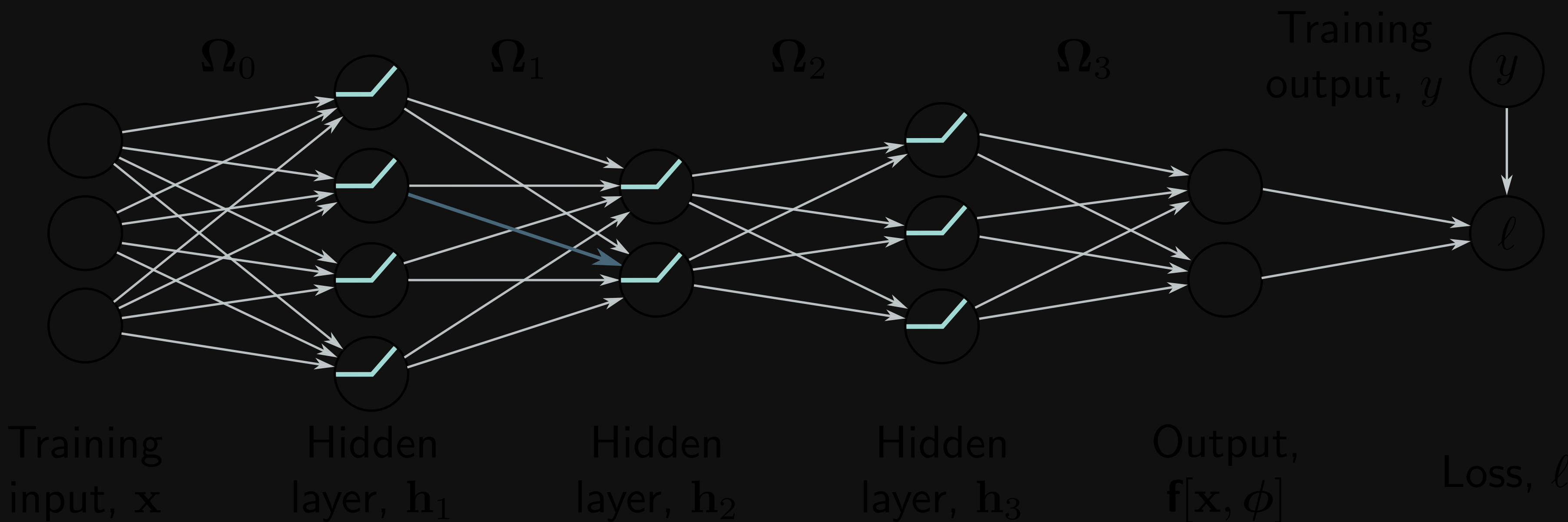


$$\frac{\partial l_i}{\partial \boldsymbol{\Omega}_k} = \frac{\partial l_i}{\partial \mathbf{f}_k} \frac{\partial \mathbf{f}_k}{\partial \boldsymbol{\Omega}_k} = \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{f}_k} \frac{\partial \mathbf{f}_{k+1}}{\partial \mathbf{h}_{k+1}} \frac{\partial l_i}{\partial \mathbf{f}_{k+1}} \mathbf{h}_k^T$$

where activation $\mathbf{h}_{k+1}$ results from pre-activation $\mathbf{f}_k$.

Forward Pass

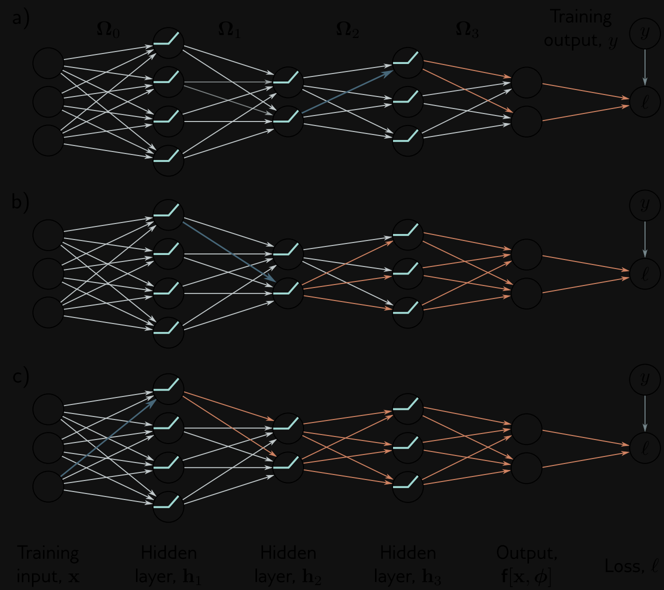
Gradients of $l_i = l(\mathbf{f}[\mathbf{x}_i, \phi], \mathbf{y}_i)$ w.r.t. weights depend on activations $\mathbf{h}_k$, which we compute first.



$$\mathbf{h}_k = \mathbf{a}(\mathbf{f}_{k-1})$$
$$\mathbf{f}_k = \beta_k + \boldsymbol{\Omega}_k \mathbf{h}_k$$

Backward Pass

Then compute gradients backwards using chain rule.



$$\frac{\partial l_i}{\partial \mathbf{f}_k} = \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{f}_k}\frac{\partial \mathbf{f}_{k+1}}{\partial \mathbf{h}_{k+1}}\frac{\partial l_i}{\partial \mathbf{f}_{k+1}} = \mathbb{I}[\mathbf{f}_k > 0] \odot \mathbf{\Omega}_{k+1}^T \frac{\partial l_i}{\partial \mathbf{f}_{k+1}}$$

$$\frac{\partial l_i}{\partial \mathbf{\Omega}_k} = \frac{\partial \mathbf{f}_k}{\partial \mathbf{\Omega}_k}\frac{\partial l_i}{\partial \mathbf{f}_k} = \frac{\partial l_i}{\partial \mathbf{f}_k}\mathbf{h}_k^T$$

Backpropagation Algorithm

Forward pass computes activations

$$
\begin{aligned}
\mathbf{f}_0 &= \beta_0 + \boldsymbol{\Omega}_0 \mathbf{x}_i \\
\mathbf{h}_k &= \mathbf{a}(\mathbf{f}_{k-1}) & k \in \{1, \ldots, K\} \\
\mathbf{f}_k &= \beta_k + \boldsymbol{\Omega}_k \mathbf{h}_k & k \in \{1, \ldots, K\}
\end{aligned}
$$

Backward pass computes gradients

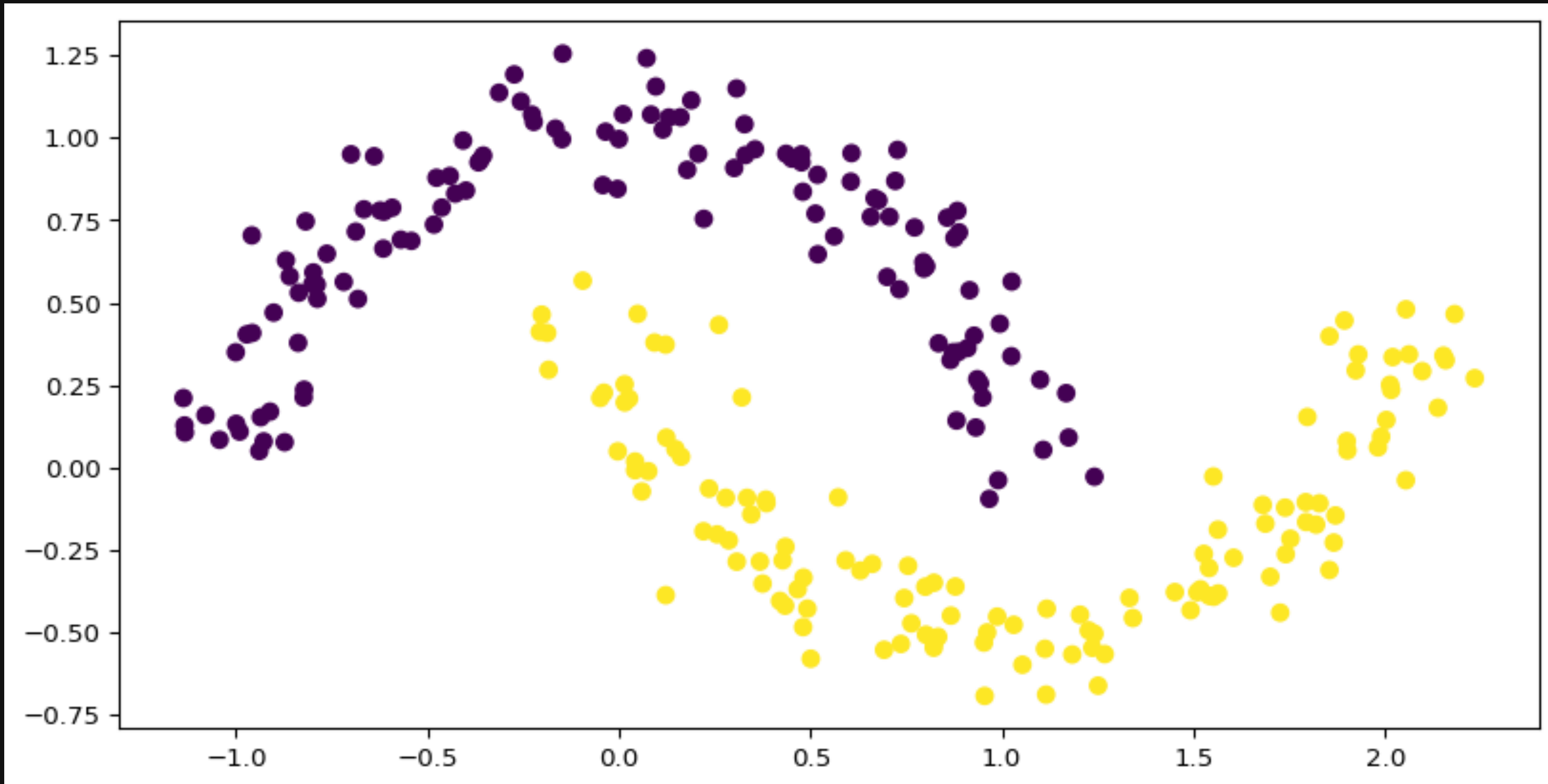
$$
\begin{aligned}
\frac{\partial l_i}{\partial \beta_k} &= \frac{\partial l_i}{\partial \mathbf{f}_k} & k \in \{K, K-1, \ldots, 0\} \\
\frac{\partial l_i}{\partial \boldsymbol{\Omega}_k} &= \frac{\partial l_i}{\partial \mathbf{f}_k} \mathbf{h}_k^T & k \in \{K, K-1, \ldots, 0\} \\
\frac{\partial l_i}{\partial \mathbf{f}_{k-1}} &= \mathbb{I}[\mathbf{f}_{k-1} > 0] \odot \left(\boldsymbol{\Omega}_k^T \frac{\partial l_i}{\partial \mathbf{f}_k} \right) & k \in \{K, K-1, \ldots, 1\}
\end{aligned}
$$

where $l_i = l(\mathbf{f}[\mathbf{x}_i, \phi], \mathbf{y}_i)$ and $\mathbf{h}_0 = \mathbf{x}_i$ (i.e. do for each training example in batch).[1]

1. This is done in parallel across a batch, so we get tensors instead of matrices.

PyTorch Implementation

Moons Dataset



Neural Network Implementation

```python
import torch
import torch.nn as nn

class MyFirstNet(nn.Module):
  def __init__(self):
    super(MyFirstNet, self).__init__()
    self.layers = nn.Sequential(
        nn.Linear(2, 16),
        nn.ReLU(),
        nn.Linear(16, 2),
    )

  def forward(self, x):
     return self.layers(x)

  def predict(self, x):
    output = self.forward(x)
    return torch.argmax(output, 1)

  def train(self, X, y):
    loss_function = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=1e-2)

    epochs = 15000
```

```
MyFirstNet(
  (layers): Sequential(
    (0): Linear(in_features=2, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=2, bias=True)
  )
```

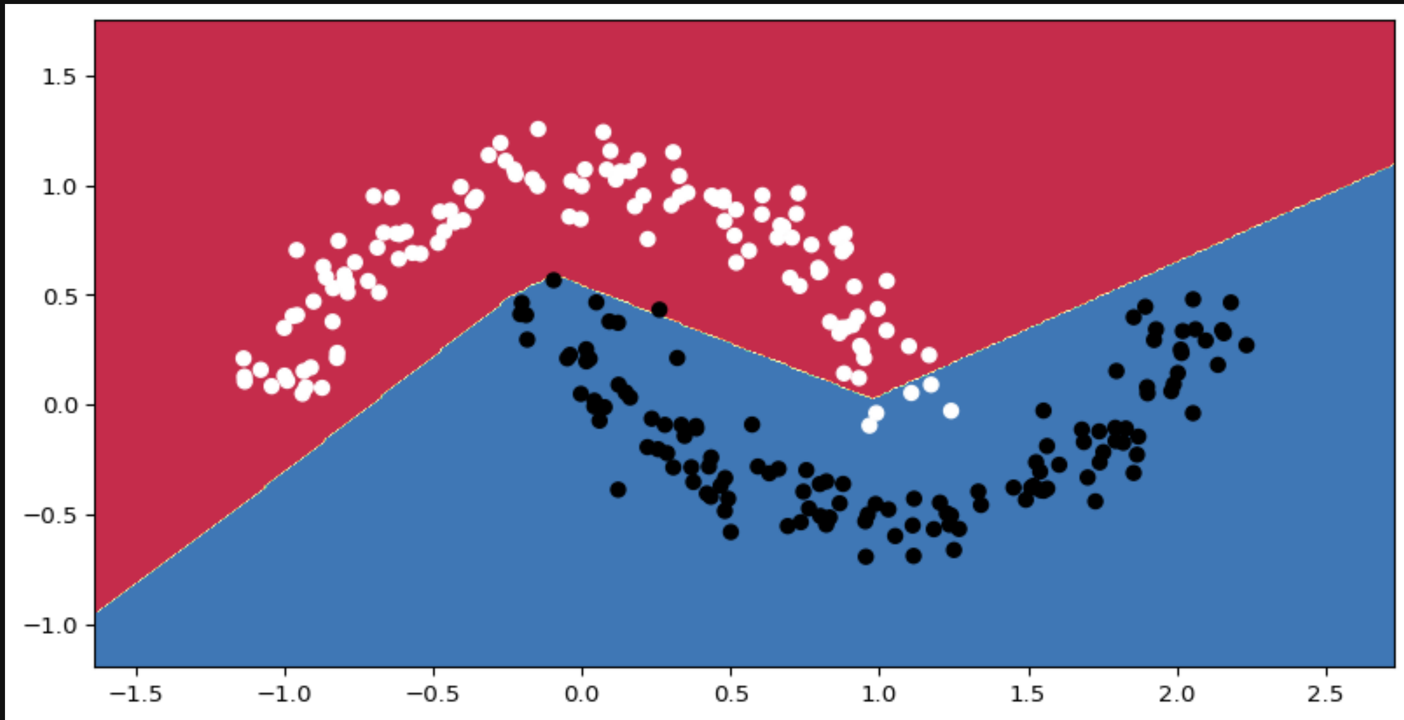
```
)
Sample input:
 tensor([[ 0.9610, -0.4991],
        [ 0.8780, -0.3601],
        [-1.0799,  0.1594],
        [ 0.8342,  0.3770],
        [ 0.5005, -0.5789]], device='mps:0')

Network output:
 tensor([[ 0.1133, -0.0112],
        [ 0.1046, -0.0326],
        [ 0.3253, -0.3412],
```

Training

```
1  losses = model.train(X, y)
```

```
Epoch 0 loss is 0.6564172506332397
Epoch 1000 loss is 0.28181493282318115
Epoch 2000 loss is 0.2414126992225647
Epoch 3000 loss is 0.2243497371673584
Epoch 4000 loss is 0.21077106893062592
Epoch 5000 loss is 0.197674959897995
Epoch 6000 loss is 0.18398109078407288
Epoch 7000 loss is 0.16948094964027405
Epoch 8000 loss is 0.15426474809646606
Epoch 9000 loss is 0.13909949362277985
Epoch 10000 loss is 0.1247006431221962
Epoch 11000 loss is 0.11129634082317352
Epoch 12000 loss is 0.09911473095417023
Epoch 13000 loss is 0.08828072249889374
Epoch 14000 loss is 0.07882219552993774
```



1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5

Another Example

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from torch.optim.lr_scheduler import StepLR

# input size, hidden layer size, output size
D_i, D_k, D_o = 10, 40, 5
# create model with two hidden layers
model = nn.Sequential(
  nn.Linear(D_i, D_k),
  nn.ReLU(),
  nn.Linear(D_k, D_k),
  nn.ReLU(),
  nn.Linear(D_k, D_o))

# He initialization of weights
def weights_init(layer_in):
  if isinstance(layer_in, nn.Linear):
    nn.init.kaiming_normal_(layer_in.weight)
    layer_in.bias.data.fill_(0.0)
model.apply(weights_init)
# choose least squares loss function
criterion = nn.MSELoss()
# construct SGD optimizer and initialize learning rate and momentum
optimizer = torch.optim.SGD(model.parameters(), lr = 0.1, momentum=0.9)
```

```
Epoch    99, loss 4.419
```

References

Prince, Simon J. D. 2023. *Understanding Deep Learning*. Cambridge, Massachusetts: The MIT Press.